
AiR Lab

코딩 세미나 5주차

2025. 9. 18.

전준

|| 목 차 ||

I . 전략 조정 1

II . Pred & Fail 3

AiR Lab 코딩 세미나 5주차

I 전략 조정

□ Agument Strength 전략

- 이전의 실험에서 Strength 전략에 cutmix/mixup에는 Strength가 적용되지 않은 문제를 확인, 이를 해결하기 위해 train에 적용되어 있는 cutmix/mixup 계수 Strength를 추가 후 재실험
- 실험 조건은 동일 환경 대비 성능 향상 정도를 확인하기 위해 40 epoch 고정
- top-1 acc/val: 87.08

model	top-1 acc
Swin_tiny + @ + strength	86.78
Swin_tiny + @ + strength(cutmix/mixup)	87.08

표1. cutmix/mixup strength 적용 모델 비교

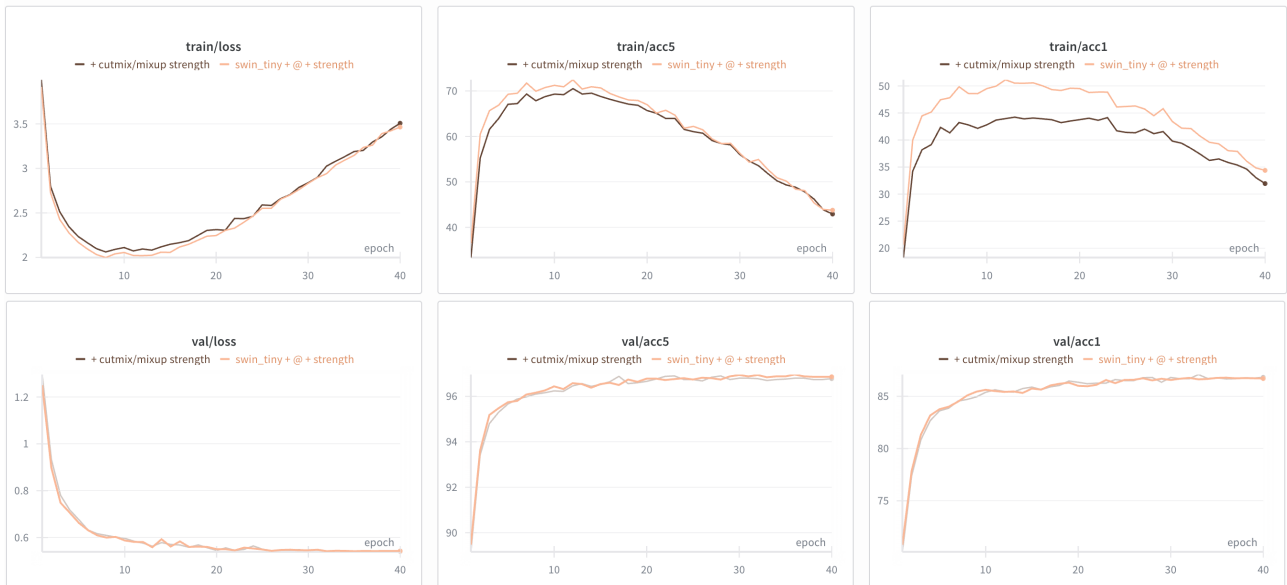


그림1. strength가 적용된 증강 성능 비교

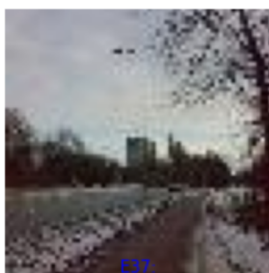
```
def train(model, dataloader, criterion, optimizer, epoch=9999, strength=1.0):
    acc1_meter = AverageMeter(name='accuracy top 1')
    acc5_meter = AverageMeter(name='accuracy top 5')
    loss_meter = AverageMeter(name='loss')
    n_iters = len(dataloader)
    cutmix_prob = 0.5 * strength
    mixup_prob = 0.5 * strength
    cutmix_alpha = 1.0 * strength
    mixup_alpha = 1.0 * strength
    그림2. cutmix/mixup strength 구현 코드 (train.py)
```

II Pred & Fail

□ Pred & Fail

- 현재까지 진행된 실험 결과를 바탕으로 모델이 어떤 이미지를 예측하기 힘들어하는 경향을 보이는지 확인하기 위해 Pred & Fail 을 실행
- 가장 높은 top-1 val/acc를 가진 epoch(37)과, 마지막(40) epoch 기준으로 예측을 가장 어려워하는 클래스 5개를 뽑고 각 이미지별로 어떤 오답을 출력하는지 예측 클래스를 percentage로 출력, 출력된 이미지와 예측 클래스를 기반으로 agumentation을 조정

Compare E37 vs E40 (Class n03976657: pole)



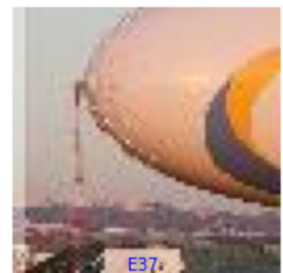
E37:
suspension bridge:66.3%
lakeside:7.6%
trolleybus:3.6%

E40:
suspension bridge:72.1%
lakeside:5.3%
trolleybus:3.5%



E37:
spider web:50.8%
backpack:13.7%
pole:3.6%

E40:
spider web:58.7%
backpack:9.0%
pole:3.3%



E37:
projectile:72.1%
water tower:3.8%
lifeboat:3.2%

E40:
projectile:76.5%
water tower:3.3%
volleyball:2.7%

Compare E37 vs E40 (Class n03400231: frying pan)



E37:
wok:93.9%
frying pan:1.5%
wooden spoon:0.1%

E40:
wok:93.0%
frying pan:1.7%
wooden spoon:0.2%



E37:
pizza:66.3%
frying pan:27.9%
wok:1.3%

E40:
pizza:62.2%
frying pan:31.3%
wok:1.2%



E37:
potpie:68.3%
frying pan:19.3%
lemon:5.5%

E40:
potpie:66.5%
frying pan:21.5%
lemon:5.5%

그림3. 예측에 실패한 이미지들 - 1

Compare E37 vs E40 (Class n04376876: syringe)



그림4. 예측에 실패한 이미지들 - 2

- 그림 3에서 frying pan이 음식에 가려지거나, pole이 사진 구석에 작게 있는 등 객체가 예측하기 힘든 위치에 있을 때 성능이 저하됨을 확인, 이를 해결하기 위해 RandomResizedCrop을 적용
- 그림 4에서 syringe이 사람손에 가려지는 등의 객체가 가려진 상황에서 오답률이 높음을 확인, 이를 해결하기 위해 이미지를 인위적으로 가지는 CoarseDropout을 적용
- RandomResizedCrop top-1 val/acc: 86.5
- CoarseDropout top-1 val/acc: 86.96
- RandomResizedCrop + CoarseDropout top-1 val/acc: 87

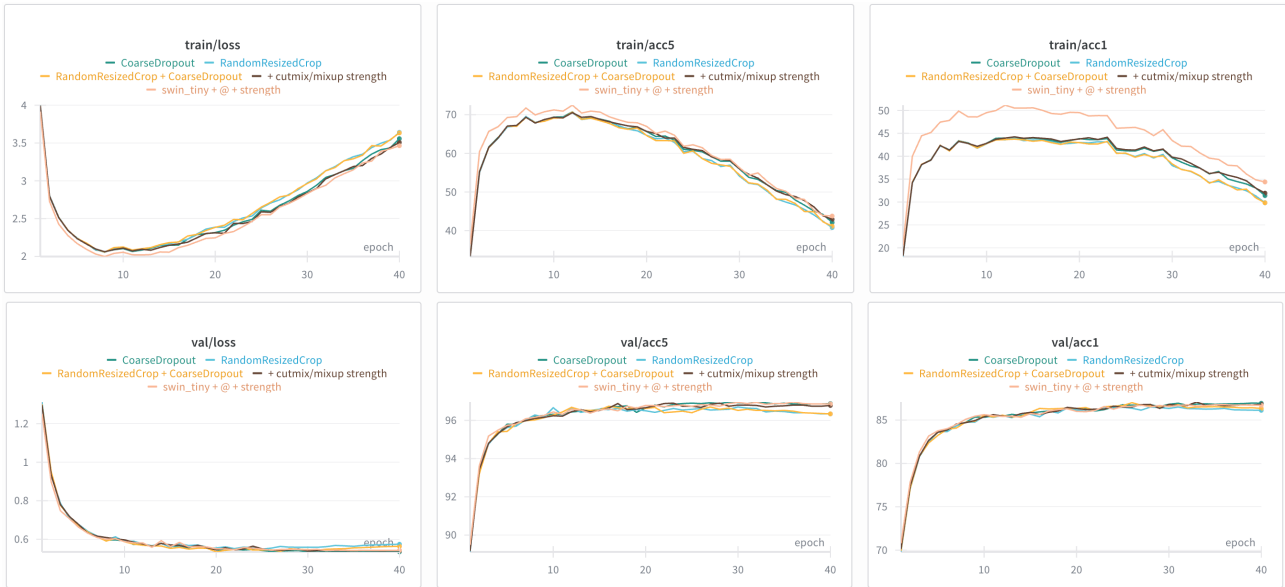


그림5. 증강 조정 전후 비교

```
def get_train_transform(strenght=1.0, img_size=224):
    return A.Compose([
        A.RandomResizedCrop(size=(img_size, img_size), scale=(0.5, 1.0), p=1.0),
        A.HorizontalFlip(p=0.5 * strenght),
        A.CoarseDropout(p=0.5 * strenght),
        A.Rotate(limit=int(15 * strenght), p=0.5 * strenght),
        A.RandomBrightnessContrast(p=0.3 * strenght),
        A.GaussNoise(var_limit=(1.0, 30.0 * strenght), p=0.3 * strenght),
        A.Normalize(mean=(0.485, 0.456, 0.406),
                     std=(0.229, 0.224, 0.225)),
        ToTensorV2()
    ])

```

그림6. 증강 조정 이후 Augmentation 코드(transforms.py)

model	top-1 acc
Swin_tiny + @ + strenght(cutmix/mixup)	87.08
Swin_tiny + @ + RandomResizedCrop	86.5
Swin_tiny + @ + CoarseDropout	86.96
Swin_tiny + @ + RandomResizedCrop + CoarseDropout	87

표2. 증강 조정 전 후 비교